

Introduction to Machine Learning

Lecture 14: Neural Networks II

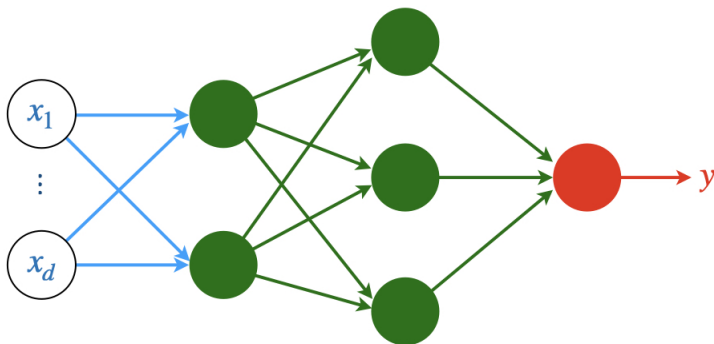
Ali Bereyhi

`ali.bereyhi@utoronto.ca`

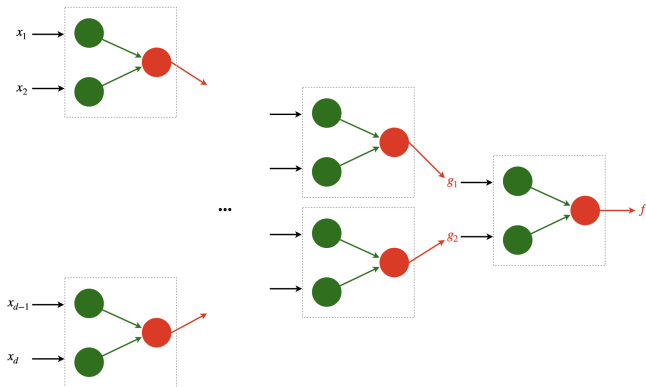
Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Quick Recap: *Neural Networks*



Quick Recap: *Expressive Power of Deep NNs*



As we go deep, our expressive power increases exponentially

Today's Agenda: *Training and Inference in NNs*

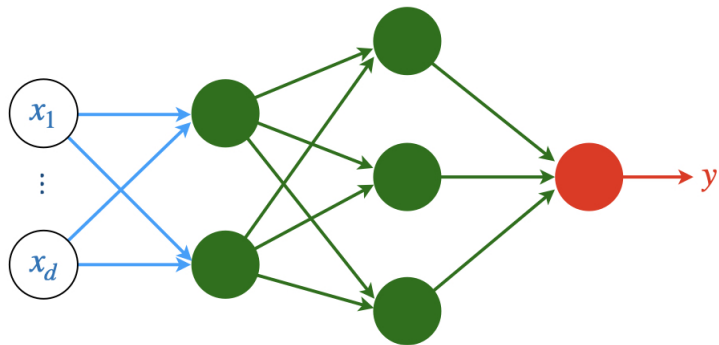
Today, we learn

How to Deploy Neural Networks

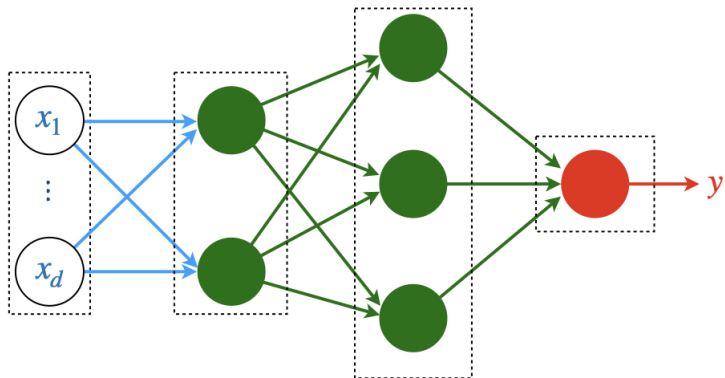
In this way, we discuss the following topics

- *Understand Inference by NNs*
- *Understand Training Loop via Gradient Descent*

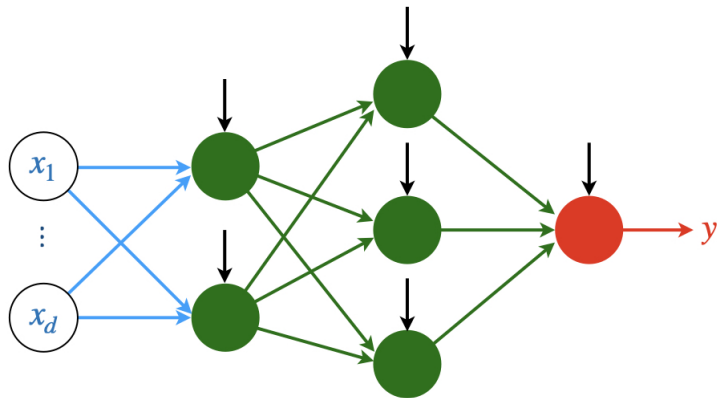
NN in a Box



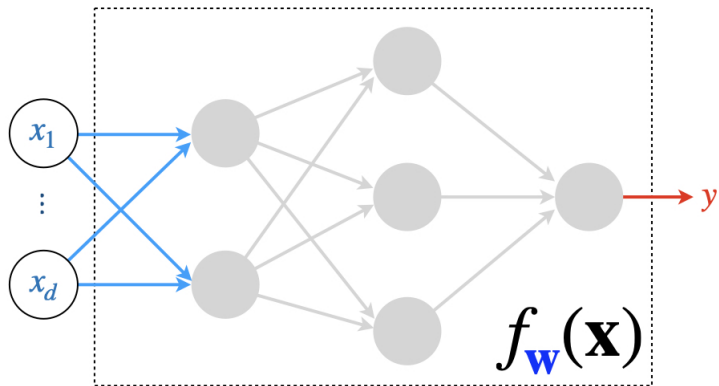
NN in a Box



NN in a Box



NN in a Box



Recall *Supervised Learning*

We have three components

- A *labeled* dataset

$$\mathbb{D} = \{(x_n \in \mathbb{X}, \mathbf{v}_n \in \mathbb{V}) : n = 1, \dots, N\}$$

- Model that relates data samples and their *labels*

$$f_{\mathbf{w}} : \mathbb{X} \mapsto \mathbb{V}$$

↳ $f_{\mathbf{w}} \in \mathbb{H}$ is now a NN with *m model parameters*, i.e.,

$$\mathbb{H} = \{f_{\mathbf{w}}(x) \text{ for all } \mathbf{w} \in \mathbb{R}^m\}$$

↳ *Hypothesis* is specified with the NN architecture

- Learning algorithm finds optimal weights for the NN

↳ This is what we call *training* a NN

Using NNs for Classification

For classification, the NN needs to give us information about the class

- *Naive approach is to set*

$$f_{\mathbf{w}}(\mathbf{x}) \in \{1, \dots, k\}$$

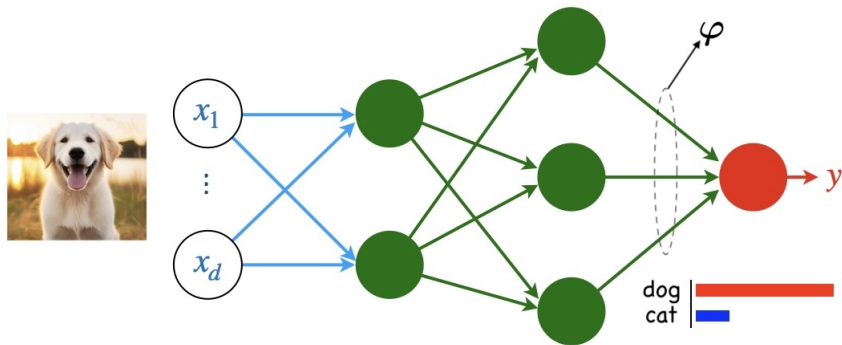
for k -class classification

↳ *This is not efficient for training as we will see*

- *In practice we set*

$$f_{\mathbf{w}}(\mathbf{x}) \propto \Pr\{\mathbf{x} \in \text{Class } i\}$$

Binary Classification via NNs



Binary Classification via NNs: Sigmoid Activation

We need a single neuron at the output layer

- Compute single linear combination and activate it by Sigmoid

$$\mathbf{w}\varphi + b = z \rightsquigarrow y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- We treat output as probability of one of the classes

$$f_{\mathbf{w}}(\mathbf{x}) = \Pr\{\mathbf{x} \in \text{Class 1}\}$$

↳ No need to compute another probability then

$$\Pr\{\mathbf{x} \in \text{Class 2}\} = 1 - f_{\mathbf{w}}(\mathbf{x})$$

- For inference, we take the class with highest probability

Multiclass Classification via NNs

? What if we have more than two classes?

! We compute the complete distribution

We need an output layer with k outputs

$$\mathbf{y} = f_{\mathbf{w}}(\mathbf{x}) = \begin{bmatrix} y_1 \\ \vdots \\ y_k \end{bmatrix} \in [0, 1]^k$$

such that $y_i = \Pr\{\mathbf{x} \in \text{Class } i\}$

$$\sum_{i=1}^k y_i = 1$$

Multiclass Classification via NNs: Softmax

Classic choice is to

- Compute k linear combinations in the output layer

$$\mathbf{W}\varphi + \mathbf{b} = \mathbf{z} \in \mathbb{R}^k$$

- Compute a Softmax of $\mathbf{z}$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_k \end{bmatrix} = \text{Soft}_{\max}(\mathbf{z}) = \frac{1}{\sum_{j=1}^k e^{z_j}} \begin{bmatrix} e^{z_1} \\ \vdots \\ e^{z_k} \end{bmatrix}$$

Softmax

Softmax computes a distribution: larger z_i has higher probability

Using NNs for Regression

For regression, the NN needs to give us real values

- *We may have a single real output*

$$\mathbf{y} = f_{\mathbf{w}}(\mathbf{x}) \in \mathbb{R}$$

- *We may have multiple real outputs*

$$\mathbf{y} = f_{\mathbf{w}}(\mathbf{x}) \in \mathbb{R}^k$$

- *We may have restricted outputs*

$$\mathbf{y} = f_{\mathbf{w}}(\mathbf{x}) \in [A, B]^k$$

Regression via NNs: Linear Output

We can consider the NN output as the real label

- *We can use directly the linear output*

$$y = z = \mathbf{w}\varphi + b \rightsquigarrow \hat{v} = y$$

↳ *The output layer is activated by linear function*

- *We may activate to restrict the output*

↳ *Say we need the output to be between -2 and 5*

$$y = \sigma(z) \rightsquigarrow \hat{v} = 7y - 2$$

Recall: Empirical Risk Minimization

Empirical Risk

Say $y_n = f_{\mathbf{w}}(x_n)$ for every $(x_n, \mathbf{v}_n) \in \mathbb{D}$; then, the empirical risk is

$$\hat{R}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}_n, \mathbf{v}_n) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_{\mathbf{w}}(x_n), \mathbf{v}_n)$$

Optimal model is the one which minimizes the empirical risk

$$\begin{aligned} \mathbf{w}^* &= \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^m} \hat{R}(\mathbf{w}) \\ &= \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^m} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_{\mathbf{w}}(x_n), \mathbf{v}_n) \end{aligned}$$

Choosing Loss: *Binary Classification*

Say we infer the label of a sample from model output y as

$$\hat{v} = \begin{cases} 1 & y \geq 0 \\ 0 & y < 0 \end{cases}$$

We can use the indicator function to compute loss

$$\mathcal{L}(y, v) = \begin{cases} 1 & y \geq 0 \text{ and } v = 0 \text{ or } y < 0 \text{ and } v = 1 \\ 0 & \text{otherwise} \end{cases}$$

But, it's not something we can easily work with! *We may also use*

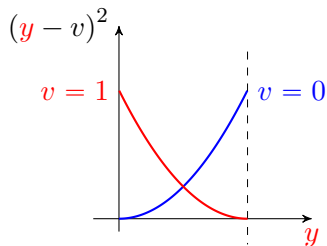
$$\mathcal{L}(y, v) = (y - v)^2$$

It still does what we want!

Binary Classification via FNN: Cross-Entropy

? What do we want?

! We want it to convince NN return right label!

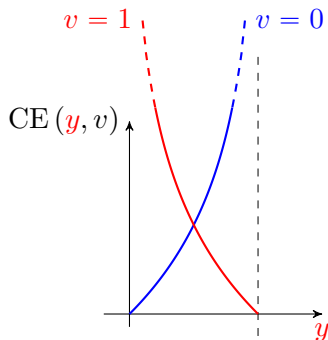


Choosing Loss: *Binary Cross Entropy*

We know y is a probability: *it's better to use maximum likelihood*

$$\mathcal{L}(y, v) = \begin{cases} -\log y & v = 1 \\ -\log(1 - y) & v = 0 \end{cases} = \text{CE}(y, v)$$

CE is again doing what we want! This time better



Choosing Loss: *Multiclass Classification*

For k -class classification, we can extend the idea of CE by setting

$$\text{CE}(\mathbf{y}, v) = -\log y_v$$

In this case, we see that

$$\begin{array}{l} 1 \rightarrow \\ 2 \rightarrow \\ \vdots \\ v \rightarrow \\ \vdots \\ k \rightarrow \end{array} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \rightsquigarrow \text{CE}(\mathbf{y}, v) = 0 \qquad \mathbf{y} \neq \begin{array}{l} 1 \rightarrow \\ 2 \rightarrow \\ \vdots \\ v \rightarrow \\ \vdots \\ k \rightarrow \end{array} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \rightsquigarrow \text{CE}(\mathbf{y}, v) > 0$$

Choosing Loss: *Generic Cross Entropy*

We can write it down more elegantly

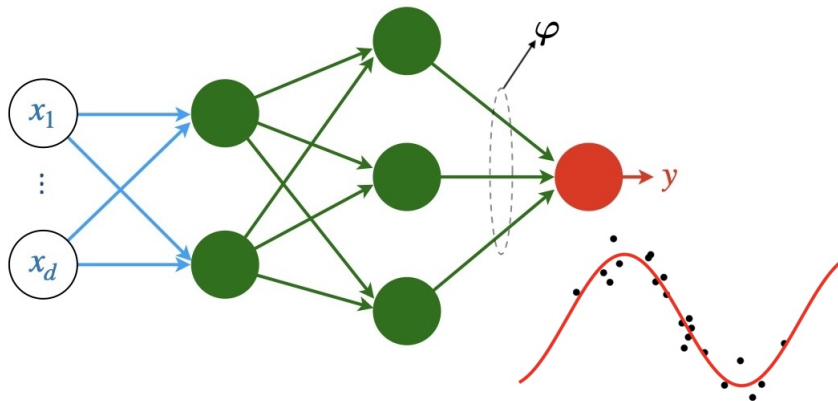
$$\text{CE}(\mathbf{y}, v) = - \sum_{i=1}^k \mathbf{1}\{i = v\} \log y_i$$

Define one-hot vector for v

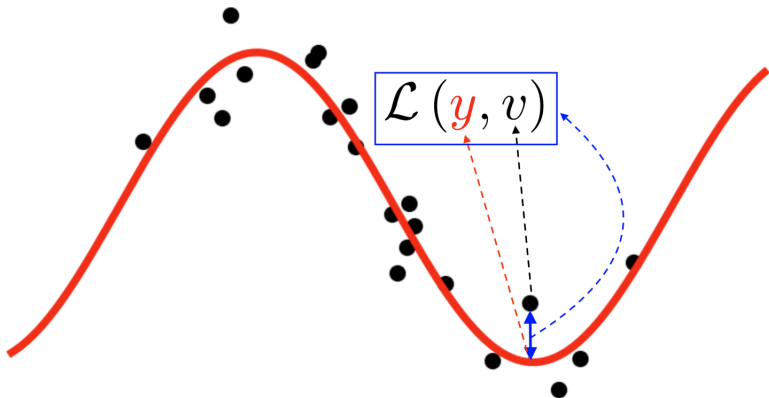
$$\mathbf{v} = \begin{matrix} 1 \rightarrow \\ 2 \rightarrow \\ \vdots \\ v \rightarrow \\ \vdots \\ k \rightarrow \end{matrix} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \rightsquigarrow \text{CE}(\mathbf{y}, v) = - \sum_{i=1}^k v_i \log y_i = \text{CE}(\mathbf{y}, \mathbf{v})$$

This is the general cross-entropy between distributions $\mathbf{y}$ and $\mathbf{v}$

Choosing Loss: *Regression*



Choosing Loss: *Regression*



Choosing Loss: *MSE*

With k -dimensional output, we can set

$$\mathcal{L}(\mathbf{y}, \mathbf{v}) = \|\mathbf{y} - \mathbf{v}\|^2 = \sum_{i=1}^k (y_i - v_i)^2$$

It again does what we want

↳ *It gets minimize when we get close to true labels $\mathbf{v}$*

Empirical risk in this case is often referred to as

Mean Squared Error $\equiv$ MSE

Recall: *Gradient Descent*

GradientDescent() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}(\mathbf{w}^t)$
- 6: Update $t \leftarrow t + 1$
- 7: **end while**
- 8: Return final $\mathbf{w}^t$

So, we need to find a way to compute $\nabla \hat{R}(\mathbf{w})$ for a given $\mathbf{w}$

Gradient of Empirical Risk

Let's define $\hat{R}_n(\mathbf{w}) = \mathcal{L}(f_{\mathbf{w}}(\mathbf{x}_n), \mathbf{v}_n)$

this is a *sample* loss, i.e., the loss computed for single sample $\mathbf{x}_n$

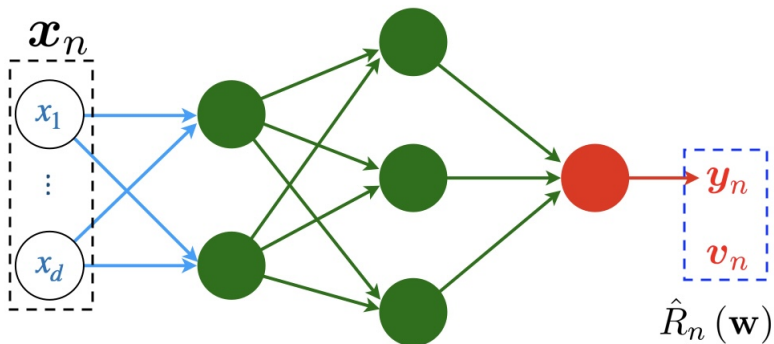
We can write

$$\begin{aligned}\nabla \hat{R}(\mathbf{w}) &= \nabla \frac{1}{N} \sum_{n=1}^N \hat{R}_n(\mathbf{w}) \\ &= \frac{1}{N} \sum_{n=1}^N \nabla \hat{R}_n(\mathbf{w})\end{aligned}$$

Conclusion

We need to find out a way to compute gradient of a sample loss

Gradient of Empirical Risk $\equiv$ Average of Sample Gradients



Further Read

- Bishop

- ↳ Chapter 4

NNs

- ESL

- ↳ Chapter 7

NNs